NagrikSetu

Technical Feasibility Proof

Explainable grievance routing and human-reviewed accountability

Item	Details
Project setting	Gwalior (prototype setting)
Prototype stack	Python • Streamlit • SQLite • scikit-learn
Evidence base	NagrikSetu Basic SRS / Implementation Plan (14 Oct 2026)
Supporting plan	Revised feature scope and dataset plan (NyayaFlow document)
Data status	Synthetic complaints only; no real municipal records
Purpose	Demonstrate feasibility, current evidence, test plan and remaining work

Important: This report separates verified prototype evidence from illustrative designs, test templates and future scope. A planned feature is not presented as a working feature.

Prepared for technical feasibility review • Version 1.0 • 15 October 2026

1. Executive summary

NagrikSetu is a local decision-support prototype for the first person who handles a civic complaint. It preserves the original text, suggests a possible department, shows evidence where available, and gives the operator a clear route to ask for clarification or leave the case for manual review. A person remains responsible for the final decision.

The SRS reports that the code was run on a 96-complaint synthetic dataset and inside the app. The reported results show that the current routing baseline is limited: the keyword baseline correctly routed 6 of 21 single-owner test complaints, while TF-IDF abstained on all 21 test cases. The compound-issue splitter flagged 3 of 3 compound cases and produced no false splits on 21 single-issue cases. These are small, synthetic test results—not proof of real-world accuracy.

This report includes the current evidence, architecture, an illustrative wireframe, pseudocode, a notebook plan, dataset exploration and API testing templates. The supplied files do not document a deployed HTTP API or completed API test run. Therefore, API test cases are provided as templates, not as passing results.

What this report can and cannot prove

Supported by the supplied SRS	Not established by the supplied files
Local Streamlit prototype and four simple pages are reported as verified.	A deployed web service, public URL or REST API.
Python, Streamlit, SQLite and scikit-learn are the stated stack.	Live municipal integration, user authentication or role-based access.
Synthetic dataset and stated test outcomes.	Performance on real complaints or independently human-labelled data.
Engines 1–6 have mixed implementation status; Engines 7–10 are not built.	A completed end-to-end implementation of every feature in the revised plan.

2. Scope and implementation status

The SRS is the source of truth for what was reported as implemented and tested. The revised feature-scope file is a roadmap; it explicitly says BUILD means planned until implemented and tested.

Component	Status in supplied SRS	How to present it
Local app and four simple pages	Reported verified	Existing prototype evidence; screenshots are mentioned in SRS but not attached here.
Keyword routing baseline	Run on full synthetic set	Working baseline, but weak test performance (6/21 correct).
TF-IDF routing	Run on full synthetic set	Model abstained on all 21 test rows; do not present as a useful router yet.
Compound-issue splitter	Tested on small synthetic test set	3/3 compounds flagged; 0/21 false splits. Small-sample result only.
Review, ledger and challenge	Reported done and verified; ledger lock included	Human decisions and audit trail are part of the current prototype claim.
Evaluation and fairness	Partly done	Human relabelling, full acceptance runner and some metrics remain.
Ownership checker	Reported done and verified	Uses an 8-row responsibility table; all ownership rows are assumptions.
Duplicate linking, safety, grounded explanations, analytics (Engines 7–10)	Not built	Roadmap only. Do not demo or claim as implemented.
REST API / HTTP endpoints	Not documented in supplied files	API templates below are proposed tests, not actual test evidence.

Prototype boundary

• Runs locally; stated stack is Python, Streamlit, SQLite and scikit-learn.

• Uses synthetic data only. No live municipal system, no login/roles, no voice, photos or maps.

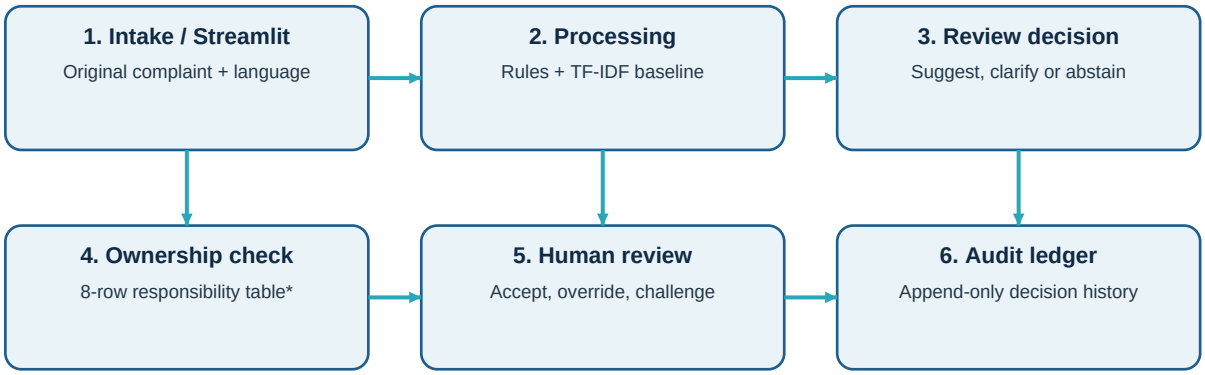
• No generative model and no retraining in V1.

• The system must not automatically close, reject or merge complaints.

• A missing location or unclear issue should stay unknown and go to a clarification/manual path.

3. System architecture

The diagram below is a simplified architecture based on the supplied SRS. It is a documentation diagram, not a claim that each planned roadmap engine is already integrated.



Local prototype: Python • Streamlit • SQLite • scikit-learn
**Ownership labels are assumptions until verified against an official source.*

Data flow in plain language

• The operator enters a complaint. The original text should remain unchanged.

• The local processing step applies the current keyword rules and TF-IDF baseline where available.

• The system may suggest an owner, flag possible multiple issues, or abstain when it cannot make a safe suggestion.

• The ownership checker compares the issue against a small responsibility table. Ownership rules are assumptions until checked against official sources.

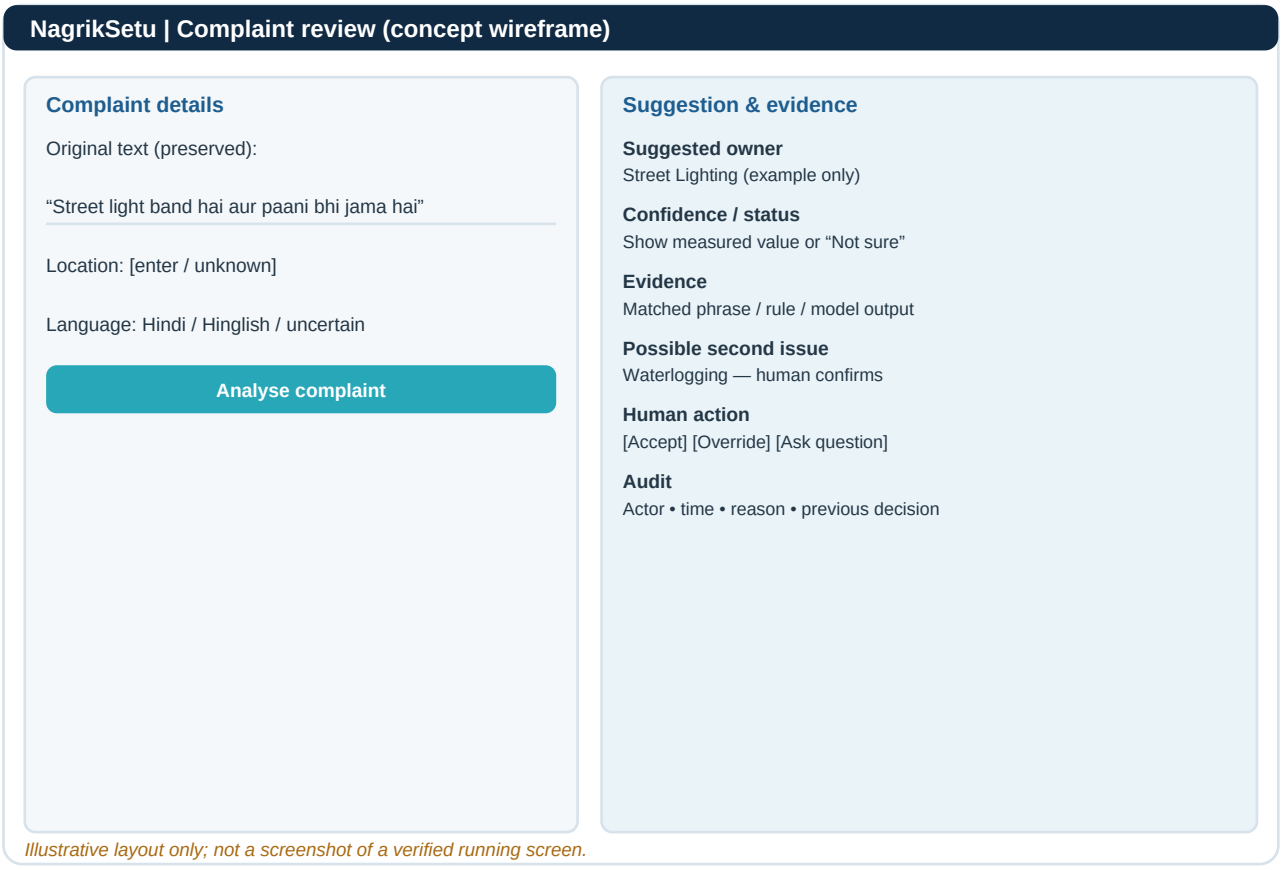
• A human accepts, overrides, asks for clarification or sends the case for manual review.

• The decision and reason are recorded in the audit trail; a correction should be added as a new row rather than erasing history.

Key design principle: a suggestion is not a final decision. The operator must be able to see uncertainty and retain control.

4. Prototype wireframe

This is a proposed screen layout for explaining the interaction during review. It is an illustrative wireframe, not a screenshot or evidence that every control is already implemented in the app.



Illustrative layout only; not a screenshot of a verified running screen.

How to explain the screen

• Left side: the operator enters the complaint and checks the location/language fields.

• Right side: the system shows a suggestion only when it has enough evidence; otherwise it shows “Not sure” and requests review.

• Evidence should show the matched phrase or rule rather than giving an unexplained label.

• If a complaint may contain two issues, the operator confirms or rejects the proposed split.

• The human action should record who made the decision and why.

Before presenting: replace this concept wireframe with a screenshot from the current Streamlit app if the submission requires proof of the actual UI. Do not label this drawing as a live screen.

5. Pseudocode

The following pseudocode describes the intended safe decision flow using features supported by the SRS. It is language-neutral and is not a copy of executable source code.

```
FUNCTION handle_complaint(original_text, location): keep original_text unchanged if location is missing: mark location as UNKNOWN ask one clarification question send to manual review record the action RETURN issue_candidates = run current keyword / TF-IDF logic split_candidates = check for possible multiple issues owner_candidates = compare issue candidates with responsibility table if issue is unclear OR confidence is low OR owners conflict: show "Not sure" and explain the uncertainty send to manual review else: show suggested owner and matched evidence ask human operator to review human_action = accept / override / ask / challenge require reason for override or transfer append decision and actor to audit ledger never auto-close, auto-reject or auto-merge RETURN case state
```

Illustrative pseudocode for a future API wrapper

```
POST /api/complaints (PROPOSED CONTRACT — not documented as implemented) Input: { "text": "...", "location": "..." } Output: { "case_id": "...", "status": "needs_review", "suggested_owner": null, "evidence": [], "clarification_question": "..." } Rules: - Validate required fields and input types. - Preserve original text. - If unsure, return needs_review; do not guess. - Keep human review as a separate action. - Log errors without exposing personal data. - Do not report this endpoint as live until it exists and is tested.
```

6. Dataset exploration

The SRS describes 96 synthetic complaints in a Gwalior setting. There is no real municipal complaint dataset in this prototype. The dataset contains English, Hindi and Hinglish variants, with related variants grouped into families so near-duplicate wording does not leak across the train/validation/test split.

Split	Rows	Role
Train	45	Used to fit or configure the baseline logic/model.
Validation	24	Used for threshold/model decisions.
Test	27	Held-out evaluation set.
Total	96	All synthetic; not real citizen submissions.

Reported evaluation results

Check	Reported result	Careful interpretation
Keyword baseline, single-owner test cases	6/21 correct; 95% interval 0.14–0.50	Low performance on a small synthetic test sample.
TF-IDF, single-owner test cases	0/21 answered; abstained on all test rows	Current threshold/model setup is not useful for routing test cases.
TF-IDF validation	Best validation score 5/18	Small validation set; not enough to claim general performance.
Compound issue splitter	3/3 compound cases flagged; 0 false splits on 21 single-issue cases	Promising in this tiny test, but more varied data is needed.
Ownership overlap	5/6 overlap cases flagged	One overlap case was missed; ownership labels are assumed.
Acceptance tests	12/13 passed in app; AT-11 code-checked only	AT-11 was not fully app-verified.

What the numbers mean

• The dataset is too small to claim real-world accuracy or fairness.

• The 23% reassignment figure in the case material is a simulated reassignment rate—not proof of a 23% misrouting rate.

• All eight rows in the current responsibility table are assumptions, not an official Gwalior department charter.

• The SRS says the complaints and labels were generated and re-checked by AI, not independently labelled by humans.

• The revised plan recommends growing toward 100–200 synthetic complaints, independent review of labels, sourced ownership rules, and frozen test data.

Dataset exploration checklist for a notebook

• Count records by split, language, issue type and owner label.

• Check duplicate/near-duplicate complaint families across splits.

• Inspect missing location, ambiguous issue and compound-issue examples.

• Review class imbalance and show denominators beside every metric.

• Keep original text and any normalized text in separate columns.

• Save dataset version, split date, assumptions and known limitations.

7. Notebook plan (Jupyter)

A notebook is useful because reviewers can see the data, reproduce the simple checks and understand why the current baseline is limited. The files do not include a notebook artifact, so this section is a build plan—not a claim that a notebook has already run.

Notebook cell/section	What to do	Evidence to save
1. Setup	Import pandas, scikit-learn and plotting tools; print versions.	Environment/version cell output.
2. Load data	Read the synthetic CSV; show shape and column names.	96 rows and expected columns, if file matches SRS.
3. Data quality	Check nulls, duplicate IDs, labels and language tags.	Counts of missing/duplicate values.
4. Split audit	Confirm family groups do not cross train/validation/test.	Pass/fail assertion and group counts.
5. Explore	Plot language, split, issue and owner counts.	Charts with labels and denominators.
6. Baseline	Run keyword baseline on the fixed test set.	Confusion matrix and correct/total count.
7. TF-IDF	Fit only on training data; choose threshold on validation.	Validation threshold and test abstention count.
8. Compound cases	Measure flagged compounds and false splits.	3/3 and 0/21 only if reproduced from the same data.
9. Error review	Print a small set of false positives, misses and abstentions.	Error table with case IDs, not personal data.
10. Conclusion	Summarize results, limitations and next experiments.	Notebook summary and reproducibility notes.

Suggested notebook folder structure

```
nagriksetu-feasibility/ data/ synthetic_complaints.csv dataset_card.md notebooks/
01_dataset_exploration.ipynb src/ routing_baseline.py ownership_rules.py tests/ test_validation.py
test_routing_contract.py evidence/ app_screenshots/ test_results/
```

Important: use the actual dataset file and rerun the notebook before adding fresh charts or result tables to the final submission. The metrics above are transcribed from the SRS, not independently reproduced by this report.

8. API testing: evidence and templates

The supplied SRS describes a local Streamlit app and does not specify HTTP endpoints, request schemas, response schemas or a running API URL. Therefore, API testing is currently a proposed next step. The table below is a test template and must not be described as completed testing.

Test ID	Test case	Expected behavior	Actual result
API-01	Valid complaint text	Return a structured response or documented validation message.	Not run — API not documented.
API-02	Empty complaint text	Reject or request clarification; no invented owner.	Not run.
API-03	Missing location	Mark location unknown; ask one question or route to manual review.	Not run.
API-04	Low-confidence / unknown issue	Abstain; do not force an owner.	Not run.
API-05	Mixed Hindi/Hinglish text	Preserve original text; allow uncertain language status.	Not run.
API-06	Compound complaint	Suggest possible sub-issues only for human confirmation.	Not run.
API-07	Invalid JSON / wrong field type	Return a clear validation error; no crash or stack trace to user.	Not run.
API-08	Human override without reason	Reject the override until a reason is supplied, if this action is exposed via API.	Not run.
API-09	Audit history	Record actor, action and reason; prior rows remain unchanged.	Not run via API.
API-10	Repeated request / duplicate submission	Do not silently merge cases; document idempotency behavior.	Not run.

Example request/response fixture (for a future endpoint)

```
Request (proposed): { "text": "Street light band hai aur road par paani jama hai", "location": "Example locality" } Expected response shape (example only, not a real response): { "status": "needs_review", "suggested_owners": [], "evidence": [], "clarification_question": "Which issue should be handled first?", "synthetic_demo": true }
```

How to turn this into real API evidence

• First decide whether an API is actually needed. A Streamlit-only prototype may not expose HTTP endpoints.

• If built, document the base URL, endpoint, method, authentication policy (if any), request schema and response schema.

• Run the tests against the actual local service and record timestamp, command/tool, status code, response body and pass/fail.

• Save sanitized request/response evidence. Do not create screenshots or status codes from imagined runs.

• If no API is implemented by submission time, mark this section “API not in current V1; test plan prepared.”

9. Prototype and evidence checklist

The SRS states that the app was run and that screenshots were kept on 14 October 2026. Those screenshots and the executable source code were not included among the two files used to prepare this report. This report therefore records the SRS claims but does not present unseen screenshots as evidence.

Artifact	What to attach for a strong submission	Status in this report
Prototype	Screenshot of the actual Streamlit home/intake screen and one processed synthetic case.	Concept wireframe included; actual screenshot not supplied.
Notebook	Run the exploration notebook and attach the .ipynb plus exported HTML/PDF or screenshots.	Notebook plan included; notebook file not supplied.
Dataset	Synthetic CSV, dataset card, split-family audit and sample rows.	Dataset statistics from SRS included; CSV not supplied.
API test	Actual endpoint specification and test report, only if an API exists.	Templates included; no actual API test results claimed.
Architecture	Diagram showing actual current components and clearly marked roadmap modules.	Simplified diagram included.
Acceptance tests	The original AT-01 to AT-13 checklist and available screenshots/logs.	Summary of reported outcomes included; source checklist not reproduced in full.
Reproducibility	Requirements file, run instructions and fixed dataset/split version.	Recommended next step.

Minimum demo sequence

• Launch the local Streamlit app using the team’s documented run command.

• Enter one clear synthetic complaint and show the suggested route and evidence.

• Enter one ambiguous or missing-location complaint and show the clarification/manual-review path.

• Show a compound complaint and explain that the human confirms any split.

• Show an override/challenge or ledger entry if this is available in the verified build.

• Explain the low baseline results honestly and identify what remains to improve.

10. Risks, limitations and next steps

Risk / gap	Why it matters	Practical next step
Small synthetic dataset	Metrics may not generalize to actual citizen complaints.	Create more diverse synthetic cases; obtain permitted public examples if possible.
No independent human labels	Labels may reflect the same assumptions as the system.	Ask a teammate not involved in writing examples to label a held-out sample.
Assumed ownership rules	Wrong owner labels can make a correct model look wrong or vice versa.	Verify against official city charters and keep a source for each rule.
TF-IDF abstains on all test cases	It currently adds no useful test-set routing coverage.	Inspect confidence/threshold logic and compare with a simple baseline on frozen data.
Limited language coverage evidence	Small sample cannot support fairness claims.	Report numerator/denominator by language; gather more examples before comparing gaps.
API not documented	Cannot claim API integration or API test success.	Build/document an endpoint only if required; test it for real.
Engines 7–10 not built	Duplicate, safety, explanation and analytics claims would be premature.	Keep these marked ROADMAP until implemented and tested.

Recommended priority order

• 1. Freeze and document the synthetic dataset and family-based split.

• 2. Fix the current baseline/abstention behavior and rerun evaluation without touching the test set during tuning.

• 3. Get independent human review of labels and verify ownership rules against public documents.

• 4. Capture actual prototype screenshots and keep a repeatable run guide.

• 5. Build a notebook that reproduces the reported metrics from the actual CSV.

• 6. Decide whether an API is needed; if yes, implement and test it. Otherwise explicitly document that V1 is Streamlit-only.

• 7. Implement roadmap engines one by one and label every feature BUILD/MOCK/ROADMAP accurately.

11. Simple explanation for the review panel

> “NagrikSetu is a local prototype that helps a grievance operator review a civic complaint. It uses a small synthetic dataset and simple keyword/TF-IDF methods to suggest a possible department. When it is uncertain, the safe choice is to ask for clarification or send the complaint for human review. The operator remains in control, and the decision history is intended to be auditable.”

If asked: “Does it work accurately?”

“The prototype has been run on 96 synthetic complaints, but the test set is small. The keyword baseline correctly routed 6 out of 21 single-owner test complaints. TF-IDF abstained on all 21 test cases. So we are not claiming high routing accuracy yet. The results help us identify what to improve next.”

If asked: “Is the API working?”

“The current SRS documents a local Streamlit app, not a deployed REST API. We prepared an API test plan, but we will only report API results after an actual endpoint is implemented and tested.”

If asked: “Are these official department rules?”

“Not yet. The current responsibility table has eight rows and every row is marked as an assumption. We need to verify the ownership rules against official Gwalior documents before using them for real complaints.”

Evidence labels to use consistently

Label	Meaning
CASE	Information supplied in the hackathon case materials.
SRS-REPORTED	Implementation or test result stated in the supplied NagrikSetu SRS; not independently rerun for this report.
ASSUMPTION	An unverified premise, including current department ownership labels.
UNKNOWN	Information not established by the supplied documents.
PROPOSED / TEMPLATE	A design or test plan that still needs implementation or execution.
ROADMAP	Future feature, not working V1.

12. Submission checklist and source notes

Before submitting, make sure the final package contains only evidence the team can actually show.

• Attach a screenshot of the running prototype, if available.

• Attach the actual synthetic CSV and dataset card.

• Attach a notebook that runs from top to bottom and produces the figures/results it reports.

• Attach actual API test results only if an endpoint exists; otherwise keep the API section labelled as a test plan.

• Keep “synthetic”, “assumption”, “unknown” and “roadmap” labels visible.

• Do not call the 23% simulated reassignment rate a misrouting rate.

• Do not claim real-world accuracy, fairness, time savings or a pilot without measurements.

Source files used

1. *NagrikSetu — Basic SRS / Implementation Plan, Round 1*, dated 14 October 2026. Used for the project scope, technology stack, synthetic dataset description, reported evaluation results, acceptance-test summary and implementation status.

2. *NyayaFlow — Revised Feature Scope + Dataset Requirements*. Used as supporting planning material for roadmap boundaries, evidence labels, dataset construction recommendations, notebook/evaluation priorities and API/feature caution. The planning document is not treated as proof that planned features are implemented.

Final note

This is a feasibility report based on the two supplied planning documents. It intentionally distinguishes existing SRS-reported evidence from proposed diagrams, notebook work and API test templates. Replace templates with actual outputs when those steps have been executed.

End of report